WAREHOUSE INVENTORY SCANNER

```java
import java.util.Scanner;

public class WarehouseInventoryScanner {
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        int[][] inventory = {
            {10, 5, 8},
            {4, -1, 12},
            {3, 6, 2}
        };

        System.out.println("WAREHOUSE INVENTORY");

        // Display inventory and skip damaged shelves
        for (int row = 0; row < inventory.length; row++) {
            for (int col = 0; col < inventory[row].length; col++) {

                if (inventory[row][col] == -1) {
                    System.out.println("Damaged shelf at Row "
                            + (row + 1) + ", Column " + (col + 1));
                    continue;
                }

                System.out.println("Row " + (row + 1)
                        + ", Column " + (col + 1)
                        + ": " + inventory[row][col] + " items");
            }
        }

        // Find the first critically low shelf
        boolean found = false;

        for (int row = 0; row < inventory.length && !found; row++) {
            for (int col = 0; col < inventory[row].length; col++) {

                if (inventory[row][col] == -1) {
                    continue;
                }

                if (inventory[row][col] <= 3) {
                    System.out.println("Critical low stock at Row "
                            + (row + 1) + ", Column " + (col + 1));
                    found = true;
                    break;
                }
            }
        }

        // Restocking with input validation
        char choice;

        do {
            System.out.print("Restock an item? (Y/N): ");
            choice = input.next().charAt(0);

            if (choice == 'Y' || choice == 'y') {
                System.out.print("Enter row (1-3): ");
                int row = input.nextInt();

                System.out.print("Enter column (1-3): ");
                int col = input.nextInt();

                if (row < 1 || row > 3 || col < 1 || col > 3) {
                    System.out.println("Invalid row or column.");
                } else if (inventory[row - 1][col - 1] == -1) {
                    System.out.println("Cannot restock a damaged shelf.");
                } else {
                    System.out.print("Enter quantity to add: ");
                    int quantity = input.nextInt();

                    if (quantity <= 0) {
                        System.out.println("Quantity must be positive.");
                    } else {
                        inventory[row - 1][col - 1] += quantity;
                        System.out.println("Restocking successful!");
                        System.out.println("Updated stock: "
                                + inventory[row - 1][col - 1]);
                    }
                }
            } else if (choice != 'N' && choice != 'n') {
                System.out.println("Invalid choice. Enter Y or N.");
            }

        } while (choice != 'N' && choice != 'n');

        // Calculate total inventory, excluding damaged shelves
        int total = 0;

        for (int row = 0; row < inventory.length; row++) {
            for (int col = 0; col < inventory[row].length; col++) {
                if (inventory[row][col] != -1) {
                    total += inventory[row][col];
                }
            }
        }

        System.out.println("Total inventory: " + total + " items");

        input.close();
    }
}
```

```
WAREHOUSE INVENTORY
Row 1, Column 1: 10 items
Row 1, Column 2: 5 items
Row 1, Column 3: 8 items
Row 2, Column 1: 4 items
Damaged shelf at Row 2, Column 2
Row 2, Column 3: 12 items
Row 3, Column 1: 3 items
Row 3, Column 2: 6 items
Row 3, Column 3: 2 items
Critical low stock at Row 3, Column 1
Restock an item? (Y/N): Y
Enter row (1-3): 2
Enter column (1-3): 3
Enter quantity to add: 4
Restocking successful!
Updated stock: 16
Restock an item? (Y/N):
```